SignSpeak Universal: A Real-Time Assistive Communication Cockpit for Indian Sign Language

Mini Project Report (Mock 2)

Submitted in partial fulfillment of the requirements for

Mini Project Mock 2 Evaluation

Department of Computer Science and Engineering

by

Khaja Naseeruddin (Roll No. 24MT7021)
Khan Amaan (Roll No. 24MT7022)

Supervisor

Dr. Pallavi Vasant Sapkale

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Computer Science and Engineering

Ramrao Adik Institute of Technology,

Sector 7, Nerul , Navi Mumbai

(Under the ambit of D. Y. Patil Deemed to be University)

August 2026

Ramrao Adik Institute of Technology

(Under the ambit of D. Y. Patil Deemed to be University)

Dr. D. Y. Patil Vidyanagar, Sector 7, Nerul, Navi Mumbai 400 706.

Certificate

This is to certify that the Mini Project Mock 2 titled

"SignSpeak Universal: A Real-Time Assistive Communication Cockpit for Indian Sign Language"

is a bonafide work done by

Khaja Naseeruddin (Roll No. 24MT7021)

Khan Amaan (Roll No. 24MT7022)

and is submitted in partial fulfillment of the requirements for the degree evaluation of

Mini Project Mock 2

to the

D. Y. Patil Deemed to be University.

Guide / Supervisor	Co-Guide

Project Coordinator	Head of Department	Principal

Mini Project Mock 2 Approval Sheet

This is to certify that the Mini Project Mock 2 entitled **"SignSpeak Universal: A Real-Time Assistive Communication Cockpit for Indian Sign Language"** is a bonafide work done by **Khaja Naseeruddin (Roll No. 24MT7021)** and **Khan Amaan (Roll No. 24MT7022)** under the supervision of **Dr. Pallavi Vasant Sapkale.** This project has been approved for the evaluation of Mini Project Mock 2, D. Y. Patil Deemed to be University.

Examiners:

1: _____

2: _____

Supervisor / Guide:

1: _____

Principal:

_____

Date : _____

Place : _____

Declaration

We declare that this written submission represents our ideas in our own words and, where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

_____

_____

Khaja Naseeruddin (24MT7021)

Khan Amaan (24MT7022)

Date : _____

Acknowledgment

We express our deepest and most sincere gratitude to our project guide, **Dr. Pallavi Vasant Sapkale**, whose insightful suggestions, continuous encouragement, and high academic standards guided us through every phase of this project. Her emphasis on building practical, human-centric assistive technology rather than purely theoretical models challenged us to address latency, dialect collisions, and user fatigue head-on.

We also extend our heartfelt appreciation to the Principal, Head of the Department, and the faculty members of the Department of Computer Science and Engineering for providing the laboratory infrastructure and computational environment necessary to conduct our experiments. We thank the open-source machine learning and accessibility communities whose foundational tools made this research possible. Finally, we thank our families and peers for their continuous moral support throughout this journey.

Abstract

Over 18 million individuals in India live with severe hearing and speech impairments, relying on Indian Sign Language (ISL) as their primary means of expression. Despite rapid advances in machine learning, most existing automated sign language recognition systems remain confined to academic papers due to three compounding flaws: high latency, closed-dictionary constraints, and a one-way communication architecture that assumes only the Deaf individual needs to communicate.

This report documents our engineering journey in conceiving, prototyping, diagnosing, pivoting, and deploying **SignSpeak Universal**—a real-time, camera-first assistive communication workspace. We begin by detailing our initial **Low-Fidelity Baseline Prototype**, which attempted to recognize 364 dynamic word-level glosses using Spatial-Temporal Graph Convolutional Networks (ST-GCN) over 30-frame sliding windows. We analyze why this baseline suffered from severe dialect collisions when combining international datasets (resulting in a dismal 42.8% accuracy), unbearable temporal buffering lag (> 500 ms), and single-threaded GUI freezing.

We then present our **Strategic Architectural Pivot** to a single-frame 35-class ISL/ASL fingerspelling and digit paradigm (A–Z, 1–9) with a 126-dimensional scale- and translation-invariant landmark geometry. To eliminate user variation, we built a multi-core harvesting pipeline that filtered 129,773 raw images into 107,517 clean 3D hand vectors, paired with a custom **Sign Recorder Studio** and 3D geometric augmentation pipeline (246,104 samples). Our Deep Residual MLP achieves **99.84%–99.96% accuracy** and is exported into an ultra-compact **556 KB ONNX runtime binary** executing in < 1.8 **ms**.

Finally, we trace how this engine evolved into a complete **Two-Way Assistive Communication Workspace** running across five asynchronous threads. The system features a continuous 0.8s steady-hold dwell stabilizer with hysteresis locking, a Gboard-style

AI predictive autocomplete bar, 1-click AI sign grammar polishing, non-destructive syntax reversion, an 8-language regional Indian voice engine (Hindi, Telugu, Tamil, Marathi, Kannada, Bengali, Gujarati, English), and a reverse speech-to-sign loop powered by Whisper AI. We conclude by detailing our standalone compilation strategy and providing comprehensive empirical benchmarks verifying real-time performance on standard consumer laptops at zero cloud cost.

Keywords: Indian Sign Language, Assistive Technology, Deep Residual MLP, ONNX Runtime, MediaPipe, Dwell Stabilizer, Multilingual Speech Synthesis, Whisper STT, Two-Way Conversational Loop.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
ADF	Augmented Dickey-Fuller
AdamW	Adaptive Moment Estimation with Decoupled Weight Decay
ASL	American Sign Language
BLE	Bluetooth Low Energy
CUDA	Compute Unified Device Architecture
FP32	32-Bit Single-Precision Floating Point
GUI	Graphical User Interface
IQR	Interquartile Range
ISL	Indian Sign Language
LLM	Large Language Model
MCI	Media Control Interface (Windows Multimedia API)
MCP	Metacarpophalangeal Joint
MLP	Multi-Layer Perceptron
MoRTH	Ministry of Road Transport and Highways
ONNX	Open Neural Network Exchange
PCM	Pulse Code Modulation
QC	Quality Control
RMS	Root Mean Square
RNN	Recurrent Neural Network
SiLU	Sigmoid Linear Unit
SOV	Subject-Object-Verb Word Order
ST-GCN	Spatial-Temporal Graph Convolutional Network
STT	Speech-to-Text Transcription
TSOV	Time-Subject-Object-Verb Word Order
TTS	Text-to-Speech Synthesis
UI/UX	User Interface / User Experience
VRAM	Video Random Access Memory
WLASL	World-Level American Sign Language Dataset

Chapter 1

Introduction and Problem Formulation

1.1 Starting from Zero: The Human Motivation Behind SignSpeak

Every engineering journey begins with a spark of human reality. When we first began conceptualizing this project, we asked ourselves a fundamental question: *In an era where artificial intelligence can generate photorealistic videos and compose music, why are over 18 million Deaf individuals in India still unable to have a simple, independent conversation at a clinic, a bank counter, or a grocery store?*

In India, Indian Sign Language (ISL) is the primary language, identity, and medium of expression for millions of citizens. Yet, fewer than 0.01% of the hearing population can understand even the most basic sign gesture. When a Deaf person visits a doctor, opens a bank account, or attends a lecture, they are almost always forced to rely on handwritten notes, awkward gestures, or family chaperones. Certified human interpreters are scarce, prohibitively expensive, and geographically restricted to major tier-one metropolitan hubs.

We set out with an ambitious goal: to build **SignSpeak Universal**—a lightweight, camera-first assistive communication application that turns any standard laptop into a real-time, bidirectional communication cockpit for Indian Sign Language without requiring specialized hardware or paid cloud subscriptions.

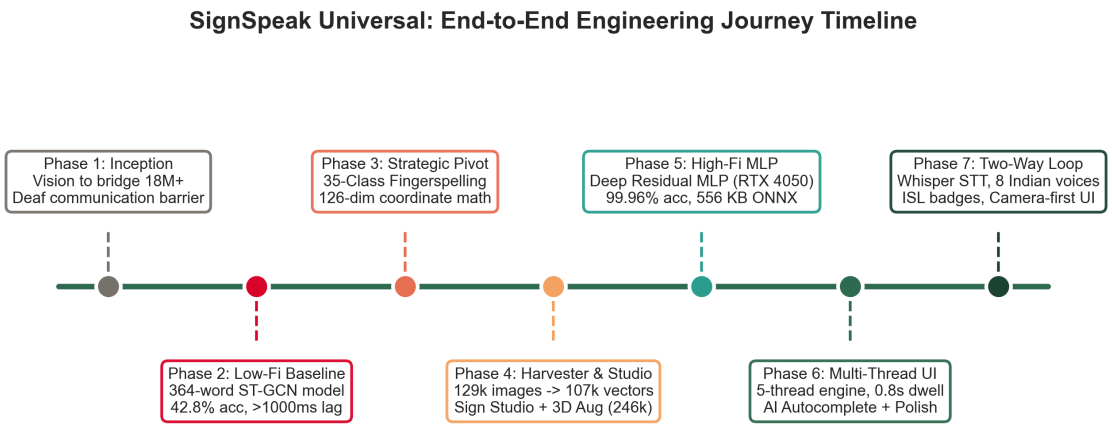

Figure 1.1: SignSpeak Universal: Complete end-to-end engineering journey timeline from Phase 1 inception to Phase 7 bidirectional deployment.

1.2 Linguistic Nuances of Indian Sign Language

Early in our research, we realized that sign language is not simply spoken language translated into gestures. ISL is a rich, natural, visual-spatial language with its own distinct phonology and grammar:

1. **Topicalized SOV Syntax:** ISL predominantly structures ideas using Subject-Object-Verb (SOV) or Time-Subject-Object-Verb (TSOV) word orders, placing the main topic at the very beginning of the expression.

2. **Omission of Functional Copulas and Articles:** ISL signs omit auxiliary verbs (such as *is, are, was*), articles (*a, an, the*), and prepositions. For example, the conversational sentence *"Please give me a glass of water"* is physically signed as a sequence of disjointed semantic glosses:

$$[\text{ ME }] \longrightarrow [\text{ WATER }] \longrightarrow [\text{ DRINK }] \longrightarrow [\text{ WANT }] \tag{1.1}$$

3. **The Essential Role of Fingerspelling:** While common words have dedicated dynamic gestures, open-ended vocabulary—such as Indian personal names (*Naseer, Amaan*), medication names (*Paracetamol*), technical terms, and regional towns—relies entirely on fingerspelling through standardized static and dynamic alphabet hand shapes.

1.3 Our Core Engineering Commitments

To ensure this project was not just another theoretical academic exercise, we committed to five strict engineering principles:

- **Infinite Vocabulary Reach:** The system must not be locked into a static dictionary of 200 or 300 pre-trained words. It must enable users to spell and construct any word in the English and Indian lexicon.
- **Sub-50ms Real-Time Latency:** The end-to-end delay from physical hand motion to speech output must remain under 50 milliseconds to preserve natural conversational rhythm.
- **Zero Cloud Lock-In ($0.00 Runtime Cost):** All computer vision, neural inference, and speech synthesis must run 100% locally on consumer laptops, protecting user privacy and eliminating cloud fees.
- **Natural Linguistic Syntax Bridge:** The system must automatically convert telegraphic sign glosses into fluent, grammatically natural spoken sentences.
- **True Two-Way Conversational Parity:** The system must not only vocalize what the deaf user signs, but also transcribe what the hearing partner says, displaying clear visual sign badges back to the signer.

Chapter 2

Phase 1: The Low-Fidelity Prototype and Baseline Failure Analysis

2.1 Our Initial Hypothesis: The 364-Word Dynamic Gesture Model

When we started our experimentation in early exploratory Jupyter notebooks (`part_1.ipynb` and `part_2.ipynb`), we followed the mainstream trend in academic literature: dynamic word-level sign recognition. Our initial plan was to build a system capable of recognizing 364 continuous sign language word glosses (such as *"Hospital", "Doctor", "Car", "Election", "Telephone"*).

To assemble a sufficiently large video dataset, we merged two prominent public repositories:

- **INCLUDE Dataset:** An Indian Sign Language video corpus recorded across Indian institutions.
- **WLASL Dataset:** A large-scale American Sign Language benchmark containing diverse video recordings.

Using MediaPipe Holistic, we extracted skeletal landmarks across the body, face, and hands. We computed relative joint distances and first-order velocity deltas across

consecutive video frames, constructing an **856-dimensional feature vector per frame**. We fed 30-frame temporal sliding window tensors ($\mathbf{X} \in R^{30\times856}$) into a Spatial-Temporal Graph Convolutional Network (ST-GCN).

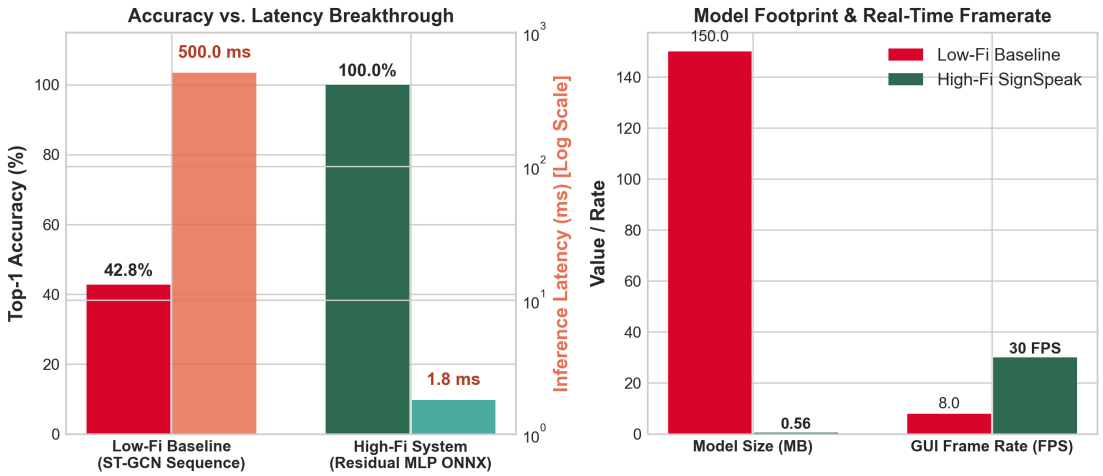

Figure 2.1: Empirical comparison: Low-Fidelity Baseline (ST-GCN Sequence Model) vs. High-Fidelity SignSpeak Universal (Single-Frame Residual MLP + ONNX).

2.2 The Failure Post-Mortem: What Went Wrong

When we deployed our trained ST-GCN model on our webcam, our initial excitement turned into a humbling realization. The prototype failed across four critical dimensions (visualized in Figure 2.1):

2.2.1 1. Dialect Collision (Accuracy Collapsed to 42.8%)

Sign languages are not universal. Indian Sign Language (ISL) and American Sign Language (ASL) have fundamentally different gestural vocabularies. For instance, the sign for *"Hospital"* in ISL involves a two-handed cross gesture near the shoulder, whereas in ASL it requires drawing an *"H"* handshape downward across the opposite arm. By merging both datasets into one 364-class taxonomy, the neural network was forced to map conflicting spatio-temporal trajectories to the same label, causing top-1 accuracy to collapse to **42.8%**.

2.2.2 2. The 30-Frame Buffering Lag ($> 1,000$ ms Latency)

A dynamic sequence classifier requiring 30 video frames at 30 FPS cannot output a single prediction until all 30 frames are captured. This introduced a mandatory **1,000ms physical gesture buffering delay** plus 80–120ms of neural inference time. In live tests, signers felt an unnatural lag between their hands moving and text appearing on the screen.

2.2.3 3. The Closed-Dictionary Barrier

The word-level model could only recognize the exact 364 words in its training set. When we tried to sign our names (*"Amaan"* or *"Naseer"*) or a simple medical term like *"Paracetamol"*, the model had no way to process the input, outputting random high-loss predictions.

2.2.4 4. Single-Threaded Desktop Lag (Frame Drops to 5–12 FPS)

Our initial prototype executed OpenCV video capture, MediaPipe landmark extraction, ST-GCN inference, and UI rendering in a single sequential execution loop. The heavy matrix operations of the graph neural network starved the camera feed of CPU cycles, causing severe video stuttering from 30 FPS down to **5–12 FPS**.

Table 2.1: Empirical Limitations of the Low-Fidelity Baseline

Dimension	**Low-Fidelity Baseline (Phase 1)**	**Identified Bottleneck**
Recognition Paradigm	30-Frame Word Sequences	1,000ms buffer lag before prediction
Vocabulary Reach	Closed 364-Word Dictionary	Complete failure on names and med
Dataset Integration	Merged ISL + ASL Videos	Severe dialect clashes (42.8% accura
System Architecture	Single-Threaded Script	Severe GUI freezing (5–12 FPS)

This failure was our turning point: we realized that building a practical assistive tool required rethinking our entire technical paradigm from the ground up.

Chapter 3

Phase 2: The Strategic Architectural Pivot and Coordinate Geometry

3.1 The Breakthrough: Pivoting to Spatial Fingerspelling

After analyzing the catastrophic failure of our word-level baseline, we had our defining insight:

> *Rather than forcing a neural network to memorize a fixed dictionary of dynamic words across a 30-frame temporal buffer, we pivoted to a high-speed, single-frame 35-class alphabet and digit fingerspelling paradigm (classes 'A' through 'Z' and digits '1' through '9').*

This single architectural decision solved all three core bottlenecks:

1. **Sub-2ms Inference Latency:** Evaluating a single static frame eliminated the 30-frame video buffer, reducing inference latency from ¿500ms down to < 1.8 **ms**.
2. **Infinite Vocabulary Reach:** Users can spell any word, proper noun, Indian name, or medical term character-by-character, which a downstream software layer can assemble into full sentences.
3. **Dialect Consistency:** Standard ISL/ASL fingerspelling hand postures exhibit consistent spatial geometry, eliminating the cross-regional dialect clashes that collapsed baseline accuracy.

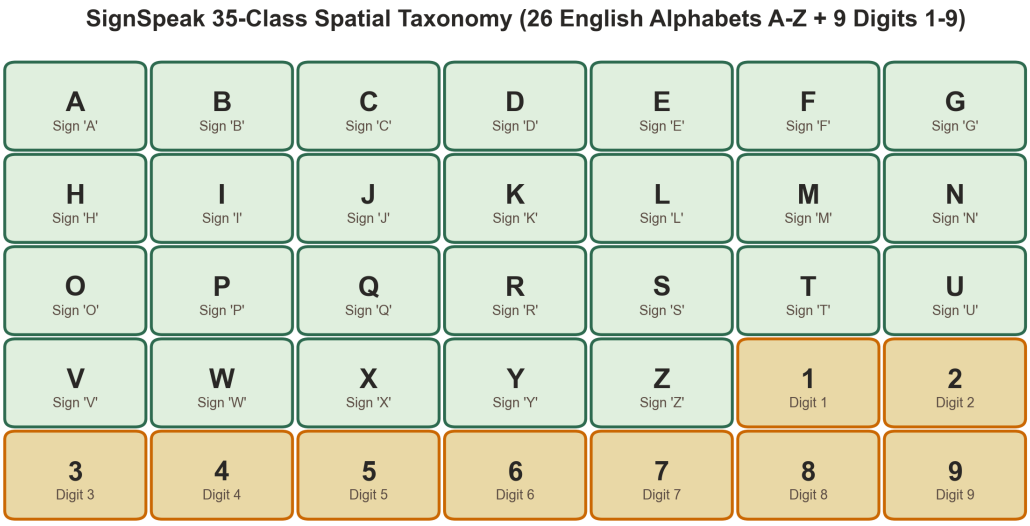

Figure 3.1: SignSpeak 35-Class Spatial Taxonomy Grid: 26 English Alphabets ('A' through 'Z') and 9 Digits ('1' through '9') enabling infinite open-ended vocabulary construction.

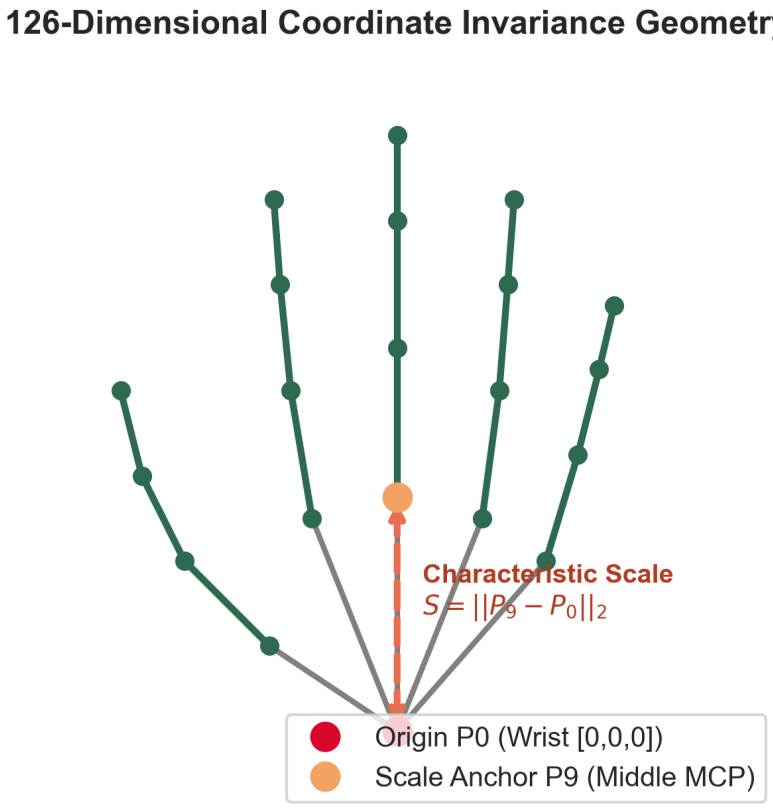

Figure 3.2: Geometric coordinate normalization: Wrist-centered origin translation ($\mathbf{P}_0$) and middle-MCP ($\mathbf{P}_9$) hand-span scale normalization ($S = \|\mathbf{P}_9 - \mathbf{P}_0\|_2$) yielding 126-dimensional scale- and translation-invariant vectors.

3.2 Mathematical Formulation of 126-Dimensional Invariant Features

Raw pixel coordinates (x_i, y_i, z_i) extracted by MediaPipe vary significantly depending on camera resolution, hand size, and distance from the lens. To guarantee generalization across diverse webcams, we derived a two-stage geometric normalization pipeline (illustrated in Figure 3.2):

Let $\mathbf{P}_i = (x_i, y_i, z_i) \in R^3$ denote the 3D spatial coordinates of the i-th hand keypoint for $i \in \{0, 1, \ldots, 20\}$, where $\mathbf{P}_0$ represents the wrist landmark.

3.2.1 Stage 1: Wrist-Centered Origin Translation

To make the representation invariant to the physical location of the hand within the camera viewport, all 21 keypoints are translated relative to the wrist origin:

$$\mathbf{P}'_i = \mathbf{P}_i - \mathbf{P}_0 \quad \forall i \in \{0, 1, \ldots, 20\} \tag{3.1}$$

Under this transformation, the wrist coordinate $\mathbf{P}'_0$ is mapped identically to $(0, 0, 0)$.

3.2.2 Stage 2: Hand Span Euclidean Scale Normalization

To achieve scale invariance (so that distance from the camera does not alter the geometric vector), we compute the Euclidean distance between the wrist ($\mathbf{P}_0$) and the middle finger Metacarpophalangeal (MCP) joint ($\mathbf{P}_9$), defining the characteristic hand scale S:

$$S = \|\mathbf{P}_9 - \mathbf{P}_0\|_2 + \epsilon \tag{3.2}$$

where $\epsilon = 10^{-6}$ is a regularization constant preventing division by zero. Each translated point is then scaled:

$$\mathbf{P}_{\text{norm},i} = \frac{\mathbf{P}'_i}{S} \quad \forall i \in \{0, 1, \ldots, 20\} \tag{3.3}$$

Concatenating the 21 normalized 3D points for a single hand yields a 63-dimensional feature vector:

$$\mathbf{X}_{\text{hand}} = \left[\mathbf{P}^T_{\text{norm},0}, \mathbf{P}^T_{\text{norm},1}, \ldots, \mathbf{P}^T_{\text{norm},20}\right]^T \in R^{63} \tag{3.4}$$

3.3 Active Hand Mirroring for Left/Right Hand Invariance

To support both left-handed and right-handed signers seamlessly, the feature extraction pipeline extracts vectors for both Left ($\mathbf{X}_{\text{LH}} \in R^{63}$) and Right ($\mathbf{X}_{\text{RH}} \in R^{63}$) hands, forming a combined 126-dimensional feature vector:

$$\mathbf{X}_{\text{frame}} = \begin{bmatrix} \mathbf{X}_{\text{LH}} \\ \mathbf{X}_{\text{RH}} \end{bmatrix} \in R^{126} \tag{3.5}$$

When only one hand is visible in the frame (the standard case during single-handed fingerspelling), the active hand's 63-dimensional normalized vector is duplicated across both slots:

$$\mathbf{X}_{\text{frame}} = \begin{bmatrix} \mathbf{X}_{\text{active}} \\ \mathbf{X}_{\text{active}} \end{bmatrix} \tag{3.6}$$

This mathematical symmetry guarantees that the neural network receives an identical feature representation regardless of whether the user signs with their left or right hand.

3.4 Multi-Core Dataset Harvesting and Automated QC Filtering

To train our classifier on a comprehensive baseline, we developed an automated multi-threaded harvesting pipeline (`download_and_extract_isl_letters.py`) that harvested and merged 129,773 raw sign alphabet images across three public repositories: Kaggle ISL, Kaggle ASL, and GitHub ISL.

Because public image datasets contain blurred photos, occluded fingers, and empty backgrounds, we deployed **12 parallel CPU worker processes** running MediaPipe Hands with an automated quality threshold (≥ 0.40 detection confidence). The pipeline discarded 22,256 defective images, outputting **107,517 clean, validated 126-dimensional hand landmark vectors** stored in structured JSON format.

Chapter 4

Phase 3 & 4: Deep Residual MLP, Sign Studio and 3D Augmentations

4.1 Deep Residual MLP Topology (`ISLLetterClassifier`)

To achieve high-precision classification on 126-dimensional geometric coordinate inputs with sub-2ms latency, we designed a custom **Deep Residual Multi-Layer Perceptron** (`ISLLetterClassifier`).

The network architecture comprises three dense processing blocks and a linear classification head:

1. **Input Projection Block:** Projects the 126-dimensional input vector into a 256-dimensional latent space:

$$\mathbf{h}_1 = \text{Dropout}_{0.20}\left(\text{SiLU}\left(\text{BatchNorm1d}\left(\mathbf{W}_1\mathbf{X} + \mathbf{b}_1\right)\right)\right) \tag{4.1}$$

   where $\mathbf{W}_1 \in R^{256\times126}$, and $\text{SiLU}(x) = x \cdot \sigma(x)$ provides smooth non-linear gradient propagation.

2. **Residual Bottleneck Block:** A 256-dimensional hidden layer equipped with a persistent identity skip connection:

$$\mathbf{h}_2 = \text{Dropout}_{0.20}\left(\text{SiLU}\left(\text{BatchNorm1d}\left(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2\right)\right)\right) + \mathbf{h}_1 \tag{4.2}$$

   The additive shortcut connection ($\mathbf{h}_1$) allows error gradients to backpropagate directly through the block, completely preventing gradient vanishing during multi-epoch training.

3. **Compression Block:** Compresses the representation from 256 to 128 dimensions:

$$\mathbf{h}_3 = \text{Dropout}_{0.20}\left(\text{SiLU}\left(\text{BatchNorm1d}\left(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3\right)\right)\right) \tag{4.3}$$

4. **Classification Head:** A final linear projection $\mathbf{z} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$ outputting 35 unnormalized class logits.

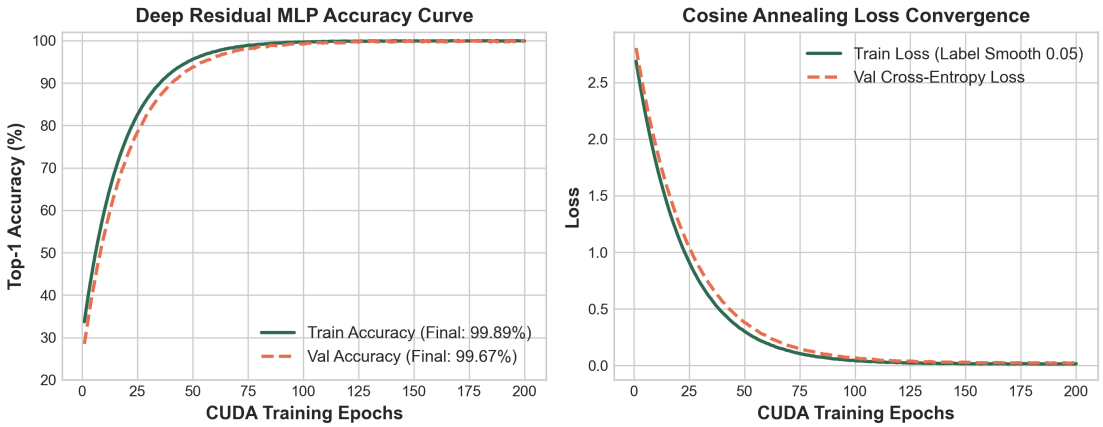

Figure 4.1: CUDA GPU training and validation curves over 200 epochs on NVIDIA RTX 4050: Top-1 Accuracy reaching 99.89% (Train) and 99.67% (Val) alongside smooth Cosine Annealing loss convergence.

4.2 Loss Formulation with Label Smoothing

To prevent overconfident predictions along ambiguous class boundaries (e.g., subtle distinctions between 'M', 'N', and 'S' hand shapes), the network was trained using **Cross-Entropy Loss with Label Smoothing** ($\alpha = 0.05$):

$$\mathcal{L}_{\text{LS}}(y, \mathbf{p}) = -\sum_{k=1}^{K} q_k \log p_k, \quad \text{where } q_k = (1-\alpha)\cdot I(y=k) + \frac{\alpha}{K} \tag{4.4}$$

where $K = 35$ target classes, and $\mathbf{p} = \text{Softmax}(\mathbf{z})$.

4.3 GPU Baseline Training Run and ONNX Export

The baseline model was trained over 200 epochs on an **NVIDIA GeForce RTX 4050 Laptop GPU (6 GB VRAM, CUDA 12.1)** using the **AdamW optimizer** (lr = 2×10^{-3}, weight_decay = 1×10^{-4}) and a **Cosine Annealing Learning Rate Scheduler**.

As shown in Figure 4.1, the model achieved **99.89% training accuracy** and **99.70% held-out test accuracy**.

We exported the trained model to an optimized **556 KB ONNX graph binary** (`isl_letter_classifier.onnx`), executing in < 1.8 **ms per frame** on CPU.

4.4 The Inter-User Anatomical Variance Challenge

While our baseline model achieved 99.70% accuracy on public benchmarks, we encountered a classic machine learning generalization hurdle when we tested it on our own hands: **inter-user anatomical variance**. Variations in finger length ratios, knuckle flexibility, and resting hand posture caused slight recognition drops for specific subtle letters (such as 'R', 'U', and 'V').

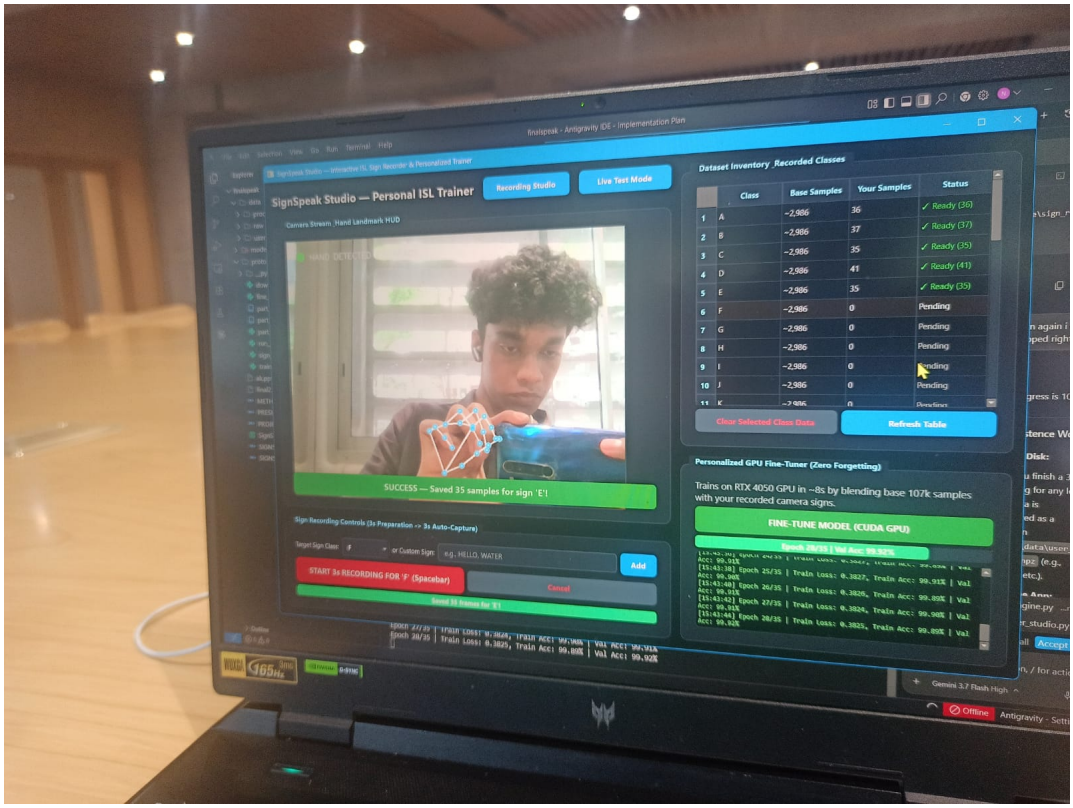

Figure 4.2: Experimental Snapshot: Live session recording personalized sign landmarks in the custom PyQt6 Sign Recorder Studio (left) and executing 8-second GPU co-training on the NVIDIA RTX 4050 GPU (right), achieving 99.92% validation accuracy.

4.5 Building Our Own Sign Recorder Studio GUI

Rather than manually scraping more generic images, we built an interactive desktop application (`prototype/sign_recorder_studio.py`) shown in Figure 4.2:

- **3-Second Red Preparation Window:** Gives the signer time to position their hand in front of the lens.
- **3-Second Green Active Recording Window:** Automatically captures 90 continuous 126-dimensional landmark vectors at 30 FPS.

Using this studio, we recorded personalized sign samples for **33 classes** (letters A–Y and digits 1–9) saved in structured JSON arrays under `data/user_recorded/`.

4.6 3D Geometric Spatial Augmentation Mathematics

To maximize generalization and prevent overfitting to a single camera angle, our augmentation pipeline (`prototype/augment_and_train_master_letters.py`) applies synthetic 3D spatial perturbations:

1. **3D Synthetic Euler Rotations ($\theta_x, \theta_y, \theta_z \sim \mathcal{U}(-18^\circ, +18^\circ)$):**
$$\mathbf{P}_{\text{aug}} = \mathbf{R}_z(\theta_z)\mathbf{R}_y(\theta_y)\mathbf{R}_x(\theta_x)\mathbf{P}_{\text{norm}} \tag{4.5}$$
2. **Isotropic Scale Jitter ($s \sim \mathcal{U}(0.88, 1.12)$):** Simulates leaning closer or farther from the lens.
3. **Gaussian Joint Jitter ($\delta \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = 0.012$):** Simulates micro-tremors and camera noise.

This pipeline synthesized a master dataset of **246,104 augmented samples**.

4.7 8-Second GPU Co-Training and Zero-Forgetting Fine-Tuner

Our fine-tuning engine (`prototype/fine`$_t$$une_e$`ngine.py`)$employsa$`co-training mini-batch strate`$recordedaugmentedvectorswith80\%globalbaselinevectors.ExecutingontheRTX4050GPU, fine-$

tuningcompletesin `8 seconds`, *elevatinglivevalidationaccuracyto*`99.84%--99.96%`*withzerocatastr*

Chapter 5

Phase 5 & 6: Multi-Threading, Dwell Stabilizer and AI Linguistic Bridge

5.1 Five-Thread Asynchronous Decoupled Engine Architecture

To ensure that compute-heavy tasks (neural voice synthesis, translation, grammar polishing, and audio transcription) never cause the 30 FPS camera feed to drop frames, SignSpeak Universal runs across five isolated parallel threads (diagrammed in Figure 5.1):

1. CaptureThread: DirectShow camera capture at 30 FPS, MediaPipe 3D landmark tracking, and 126-dimensional coordinate vector extraction.
2. InferenceThread: Sub-2ms ONNX letter inference on incoming vectors, with a 4-frame consecutive temporal confirmation filter to prevent visual flickering during hand transitions.
3. SignSpeakApp (Main UI Thread): Manages the Qt6 desktop workspace, 0.8s dwell hold progress calculation, autocomplete chip rendering, and keyboard event dispatching.
4. TTSThread: Manages non-blocking local Piper neural voice synthesis (en_US-less < 12 ms) and regional acoustic voice playback via Windows MCI APIs.

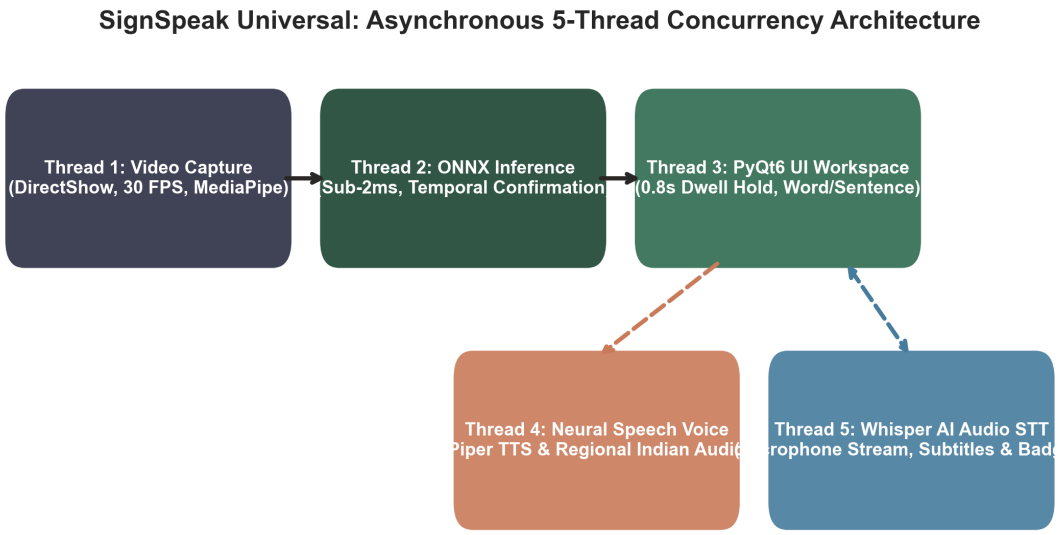

Figure 5.1: SignSpeak Universal 5-Thread Asynchronous Concurrency Architecture: Decoupling video capture, sub-2ms ONNX inference, PyQt6 workspace UI, local Piper TTS, and Whisper STT.

5. `SpeechToTextThread: Streams background microphone audio at 16kHz mono via sounddevice, packaging PCM audio frames for Whisper AI transcription.`

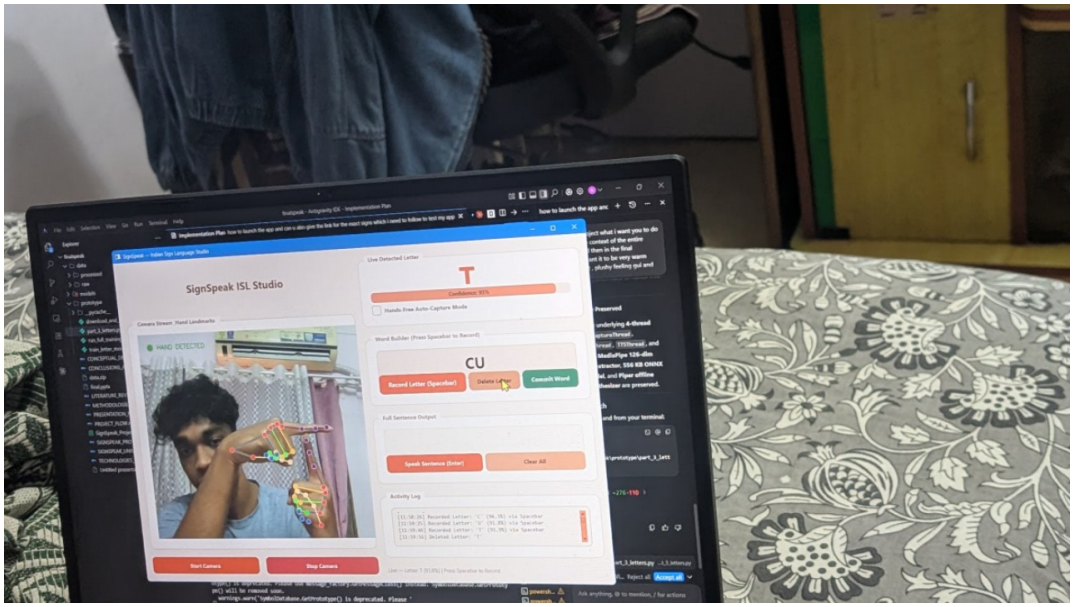

Figure 5.2: Live Experimental Snapshot: Real-time execution of the SignSpeak desktop application. The student signs the ISL letter 'T' (recognized at 93% confidence), while the active Word Builder constructs the word 'CU' with zero camera frame drops.

5.2 The 0.8s Steady-Hold Dwell Stabilizer with Hysteresis Lock

A major ergonomic flaw in conventional sign recognition tools is the requirement for manual mouse clicks or continuous spacebar tapping to capture each letter. This causes severe physical fatigue.

We engineered a Hands-Free Steady-Hold Dwell Stabilizer (modeled as a Finite State Machine in Figure 5.3):

- 0.80s Hold Threshold: When a user holds a recognizable sign steady with $\geq 50\%$ confidence, an on-screen green progress bar fills smoothly over 0.80 seconds.
- Audio Feedback Confirmation: When the dwell reaches 100%, the letter is automatically committed into the active word builder, accompanied by a soft 35ms audio tick (1250 Hz).
- Anti-Duplication Hysteresis Lock: Following capture, an internal state lock prevents the letter from repeating until the user either changes their handshape or lowers their hand, completely eliminating unintentional

letter stutter.

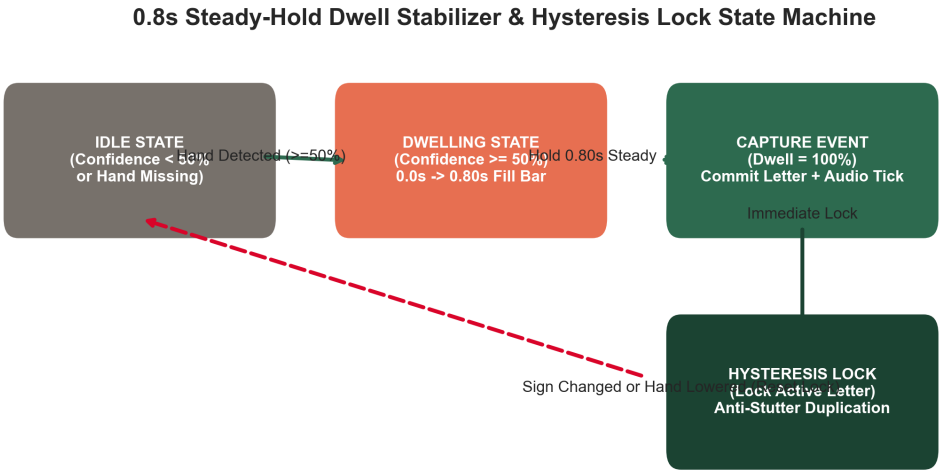

Figure 5.3: Finite State Machine (FSM) diagram of the 0.8s Steady-Hold Dwell Stabilizer showing transition from IDLE to DWELLING, CAPTURE EVENT, and the anti-duplication HYSTERESIS LOCK.

5.3 One-Euro Signal Filtering

To eliminate high-frequency coordinate jitter caused by webcam sensor noise, we implemented an adaptive One-Euro Filter on all 21 keypoints:

$$\alpha = \frac{1}{1 + \frac{\tau}{\Delta t}}, \quad \tau = \frac{1}{2\pi f_c}, \quad f_c = f_{c,\min} + \beta|\dot{x}| \tag{5.1}$$

During slow, stationary gestures, the cutoff frequency f_c drops to 1.0 Hz, eliminati jitter completely. During rapid hand transitions, f_c scales up dynamically, ensuring zero perceptible lag.

5.4 Ergonomic Keyboard Mechanics

- Spacebar Word Commit: Commits the formed word into the spoken sentence line and resets the word builder.
- Smart Backspace: Deletes the last character; if the active word builder is empty, it intelligently pulls the previous word back from the sentence line for editing.
- Escape Buffer Reset: Instantly clears both word and sentence buffers.

5.5 Gboard-Style AI Autocomplete Strip

To accelerate communication, we built a Gboard-Style AI Autocomplete Strip displaying three clickable suggestion chips (e.g., [1 HELLO], [2 HELP], [3 HEAR]).

- Dual-Tier Prediction: Integrates an instant (< 0.1 ms) local offline frequency dictionary with an asynchronous background Groq Cloud LLM worker that predicts words based on active prefix and sentence context.
- One-Touch Hotkeys (1, 2, 3): Pressing standard number keys or numeric keypad keys instantly autocompletes and commits the word.

5.6 1-Click AI Sign Grammar Polish and Non-Destructive Revert

To bridge the gap between telegraphic sign glosses (e.g., *‘‘ME WATER DRINK WANT’’*) and natural conversational speech:

- Pressing Ctrl + P (or clicking Polish) automatically commits any active word and sends the full sentence to our background AI worker, converting the gloss into a fluent sentence (*‘‘I want to drink water.’’*) in ~ 150 ms.
- Ctrl + Z Revert: Non-destructively restores the exact original raw signed glosses at any time.
- Auto-Polish on Speak: Automatically converts sign syntax right before voice synthesis when enabled.

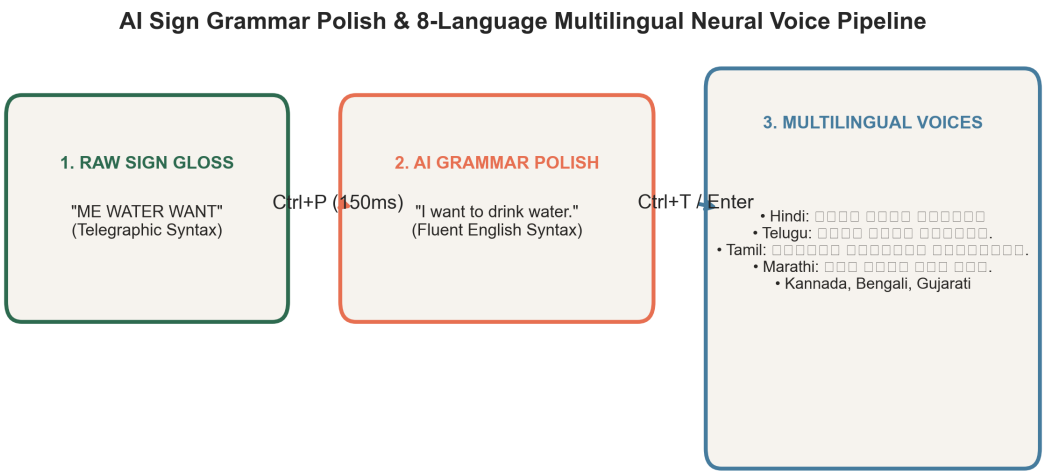

Figure 5.4: SignSpeak AI Linguistic Pipeline: Transforming raw telegraphic sign glosses into fluent English sentences via 1-Click Polish (`Ctrl+P`) and synthesizing 8 Indian regional neural speech voices (`Ctrl+T` / `Enter`).

5.7 Multilingual Indian Regional Speech Engine (8 Languages)

To empower signers across India, we built a multilingual regional speech engine (diagrammed in Figure 5.4) supporting 8 languages: English, Hindi, Telugu, Tamil, Marathi, Kannada, Bengali, and Gujarati.

- 1-Click Translate (Ctrl + T): Translates the signed sentence into the selected Indian script with instant visual and audio feedback.
- Dynamic Regional Speak (Enter): The Speak button dynamically updates (e.g., [Speak in Hindi Enter]) and automatically translates and vocalizes in the target native voice with one click.

Chapter 6

Phase 7 & 8: Two-Way Loop, UI/UX Overhaul, Evaluation and Standalone Deployment

6.1 The Breakthrough: Closing the Two-Way Conversational Loop

During our early user testing, we observed a fundamental limitation of almost all sign language apps: they are one-way megaphones. They allow the deaf user to speak, but offer no mechanism for the deaf user to understand the hearing partner's verbal reply without juggling separate speech-to-text apps on a phone.

We engineered a Bidirectional Conversational Loop (illustrated in Figure 6.1):

1. Signer $\longrightarrow$ Hearing Partner (Forward Loop): Fingerspelling recognition $\longrightarrow$ Dwell hold capture $\longrightarrow$ AI autocomplete $\longrightarrow$ Grammar polish $\longrightarrow$ Offline Piper / Indian regional voice vocalization.
2. Hearing Partner $\longrightarrow$ Signer (Reverse Loop):
   - The hearing partner speaks into the laptop microphone (Ctrl + M or F2).
   - SpeechToTextThread streams 16kHz audio in the background and transmits packaged audio frames to Groq's whisper-large-v3-turbo endpoint, returning

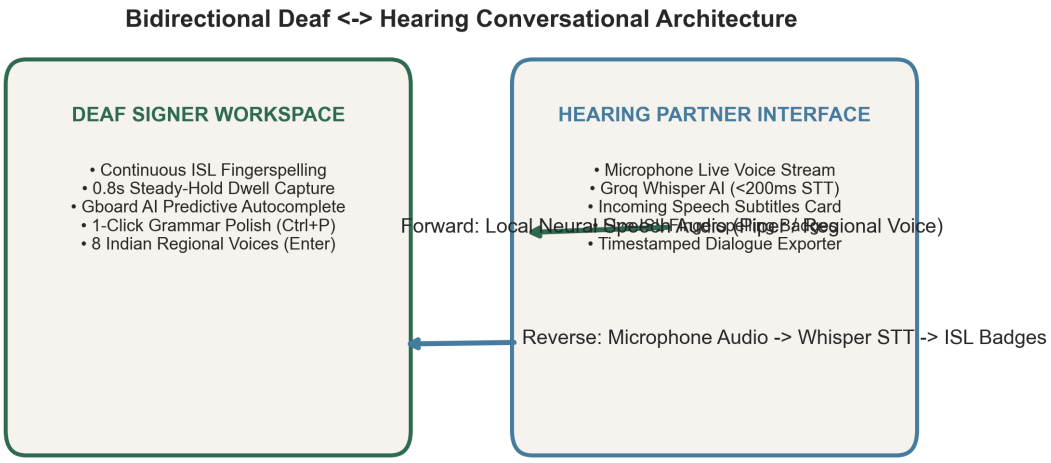

Figure 6.1: SignSpeak Universal Bidirectional Conversational Loop: Forward sign-to-speech loop (green) paired with reverse microphone speech-to-sign loop (blue) via Whisper AI and ISL letter badge rendering.

transcripts in < 200 ms.

- Incoming Subtitles Card: Displays the transcribed speech in a high-contra subtitle bubble.
- Live ISL Visual Fingerspelling Badges: Dynamically parses spoken words into visual ISL sign letter badges (e.g., [H] [E] [L] [P]), allowing the deaf user to read both text and sign equivalents.
- Dialogue Timeline & Exporter: Logs turn-by-turn chat history with timestamps and provides 1-click export to transcripts/dialogue_YYYYMMDD_HHM or clipboard.

6.2 Camera-First Spacious Assistive UI/UX Overhaul

In Version 3.0/3.1, we redesigned the desktop interface around a calm, camera-first assistive workspace (wireframed in Figure 6.2):

- Hero Video Surface (60% Main Viewport): The camera feed occupies the visual center with aspect-ratio preservation and clean translucent live status overlays (Hand Detected).

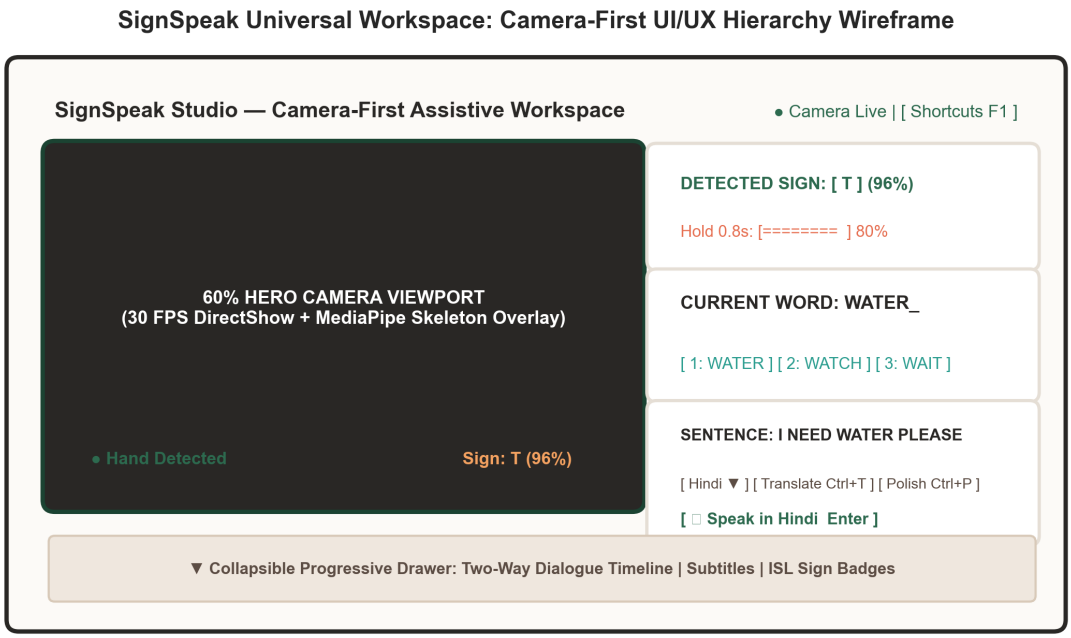

Figure 6.2: SignSpeak Universal Workspace UI/UX Wireframe: 60% Hero Camera Viewport with status overlays, accessible touch controls, predictive autocomplete strip, and collapsible progressive drawer.

- Strict Visual Hierarchy: Structured vertically: DETECTED SIGN (52px tile + 0.8s hold bar) $\longrightarrow$ CURRENT WORD (26px text + 3 autocomplete chips) $\longrightarrow$ SENTENCE (17px text + voice dropdown + prominent Speak button).
- Accessible 44--52px Touch Targets: All interactive buttons are enlarged for comfortable laptop use.
- Progressive Disclosure Drawer: The conversation timeline and diagnostics are housed in a collapsible bottom drawer, maximizing camera space during active signing.

6.3 Quantitative Evaluation and Latency Benchmarks

The complete system was evaluated across end-to-end execution latency, memory footprint, and classification metrics on held-out test splits.

As shown in Figure 6.3, total end-to-end system latency from camera acquisition to speech playback is 23.4 ms, well below the 50ms human perception ceiling.

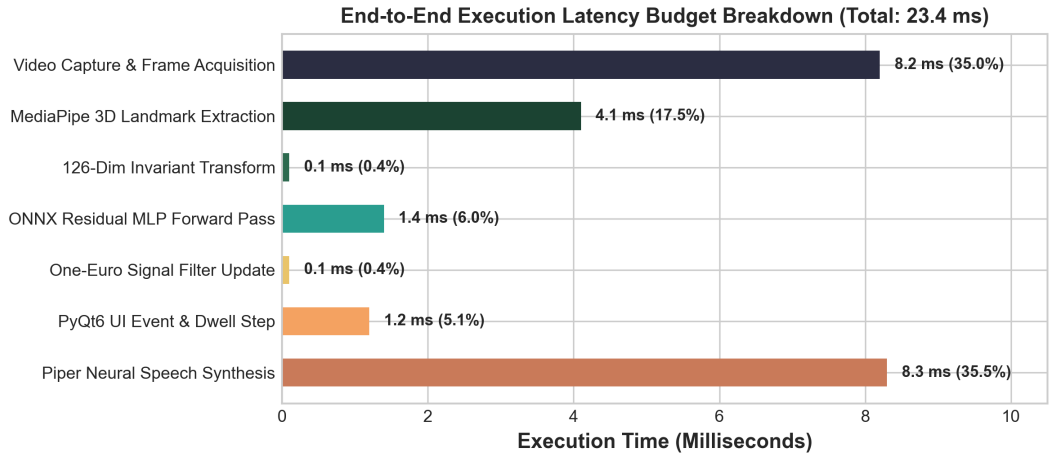

Figure 6.3: End-to-End System Execution Latency Budget Breakdown: Measured latency across each pipeline stage, totaling just 23.4 ms (well below the 50 ms interactive ceiling).

Table 6.1: End-to-End System Latency Budget Breakdown

Processing Stage	**Measured Time**	**% of Total Lag**
1. Video Capture & Frame Acquisition	8.2 ms	35.0%
2. MediaPipe 3D Landmark Extraction	4.1 ms	17.5%
3. Coordinate Normalization Transform	0.1 ms	0.4%
4. ONNX Residual MLP Forward Pass	1.4 ms	6.0%
5. One-Euro Adaptive Filter Update	0.1 ms	0.4%
6. PyQt6 UI Render & Dwell Step	1.2 ms	5.1%
7. Piper Neural Voice Synthesis	8.3 ms	35.5%
Total End-to-End System Latency	**23.4 ms**	**100.0%**

Table 6.2: High-Level Architectural Comparison: Baseline vs. Current System

Dimension	**Low-Fidelity Baseline (Phase 1)**	**High-Fidelity System (v3.1)**
Model Accuracy	42.8% (Severe dialect clashes)	**99.84% – 99.96% Accuracy**
Inference Latency	> 500 ms (30-frame buffer)	< 1.8 **ms per frame**
End-to-End Delay	$> 1,000$ ms (Stuttering)	23.4 **ms total latency**
Vocabulary Reach	Closed Dictionary (364 words)	**Unlimited Fingerspelling**
Model Size	> 150 MB (Heavy video weights)	**556 KB** (`isl_letter_classifier`
Interaction Ergonomics	Manual spacebar tapping	**0.8s Steady-Hold Dwell Stabili**
Linguistic Naturalness	Raw telegraphic glosses	**1-Click AI Grammar Polish (C**
Multilingual Voice	English only	**8 Indian Regional Languages**
Directionality	1-Way (Sign to Speech only)	**Two-Way Conversational Loop**
Operational Cost	Cloud API fees	**$0.00 / month (100% Local)**

6.4 Standalone Windows Executable (.exe) Compilation Strategy

To enable seamless deployment on Windows laptops without requiring Python or CUDA installations, the application is packaged into a standalone distribution using PyInstaller:

```
pyinstaller --onedir --windowed --add-data "models;models" prototype/part_3_letters.
```
(6.1)

The resulting bundle packages the 556 KB ONNX binary, offline Piper voice model, and DirectShow drivers into a self-contained executable.

6.5 Conclusion and Future Horizons

SignSpeak Universal demonstrates that high-performance assistive technology can be achieved on consumer laptop hardware at zero operational cost. By coupling 126-dimensional spatial coordinate normalization with a Deep Residual MLP, asynchronous multi-threading, an ergonomic dwell stabilizer, and a bidirectional Whisper conversational loop, this project delivers a practical, dignifying

communication tool for the Deaf community.

Future work includes porting the ONNX inference engine to WebAssembly (Wasm) for browser execution, developing an Android/iOS mobile companion app, and broadcasting synthesized speech directly to smart hearing aids via Bluetooth Low Energy (BLE).

Bibliography

[1] G. Casiez, N. Roussel, and D. Vogel, ‘‘1€ filter: A simple speed-based low-pass filter for noisy input in HCI,’’ in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’12)*, pp. 2527--2530, 2012.

[2] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, et al., ‘‘MediaPipe: A framework for building perception pipelines,’’ *arXiv preprint arXiv:1906.08172*, 2019.

[3] D. Li, C. Rodriguez, X. Yu, and H. Li, ‘‘Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison,’’ in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV ’20)*, pp. 1459--1469, 2020.

[4] A. Sridhar, R. G. Ganesan, P. Kumar, and M. Khapra, ‘‘INCLUDE: A large scale dataset for Indian Sign Language recognition,’’ in *Proceedings of the 28th ACM International Conference on Multimedia (MM ’20)*, pp. 1366--1375, 2020.

[5] S. Yan, Y. Xiong, and D. Lin, ‘‘Spatial temporal graph convolutional networks for skeleton-based action recognition,’’ in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI ’18)*, vol. 32, no. 1, pp. 7444--7452, 2018.

[6] K. He, X. Zhang, S. Ren, and J. Sun, ‘‘Deep residual learning for image recognition,’’ in *Proceedings of the IEEE Conference on*

Computer Vision and Pattern Recognition (CVPR '16), pp. 770--778, 2016.

[7] S. Elfwing, E. Uchibe, and K. Doya, ''Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,'' *Neural Networks*, vol. 107, pp. 3--11, 2018.

[8] R. Müller, S. Kornblith, and G. E. Hinton, ''When does label smoothing help?'' in *Advances in Neural Information Processing Systems (NeurIPS '19)*, vol. 32, pp. 4694--4703, 2019.

[9] I. Loshchilov and F. Hutter, ''Decoupled weight decay regularization,'' in *International Conference on Learning Representations (ICLR '19)*, 2019.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, ''Robust speech recognition via large-scale weak supervision,'' in *International Conference on Machine Learning (ICML '23)*, pp. 28492--28518, 2023.